

Homework 3 DQN & Variants

Q&A and Results Showcase

? Q1: HW3-1 Naive DQN for static mode

Question / Task:

Run the provided code naive or Experience buffer replay. Chat with ChatGPT about the code to clarify your understanding. Submit a short understanding report.

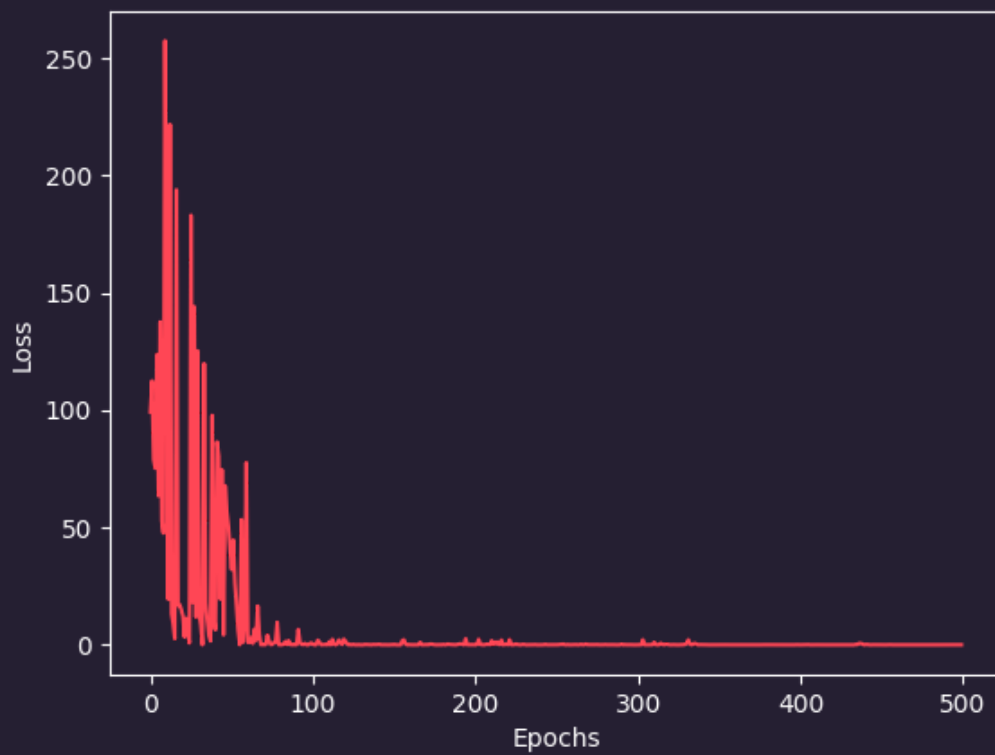
A1 (Understanding Report):

In `static` mode, all objects are fixed. The state is represented as a flattened 64-dimensional array.

- **Naive DQN:** Trains strictly online. Since consecutive frames are highly correlated, the learning process oscillates heavily and struggles to stabilize.
- **Experience Replay:** Solves instability by storing past experiences in a buffer and sampling mini-batches randomly. This breaks temporal correlation, drastically improving the smoothness and convergence of the loss curve.

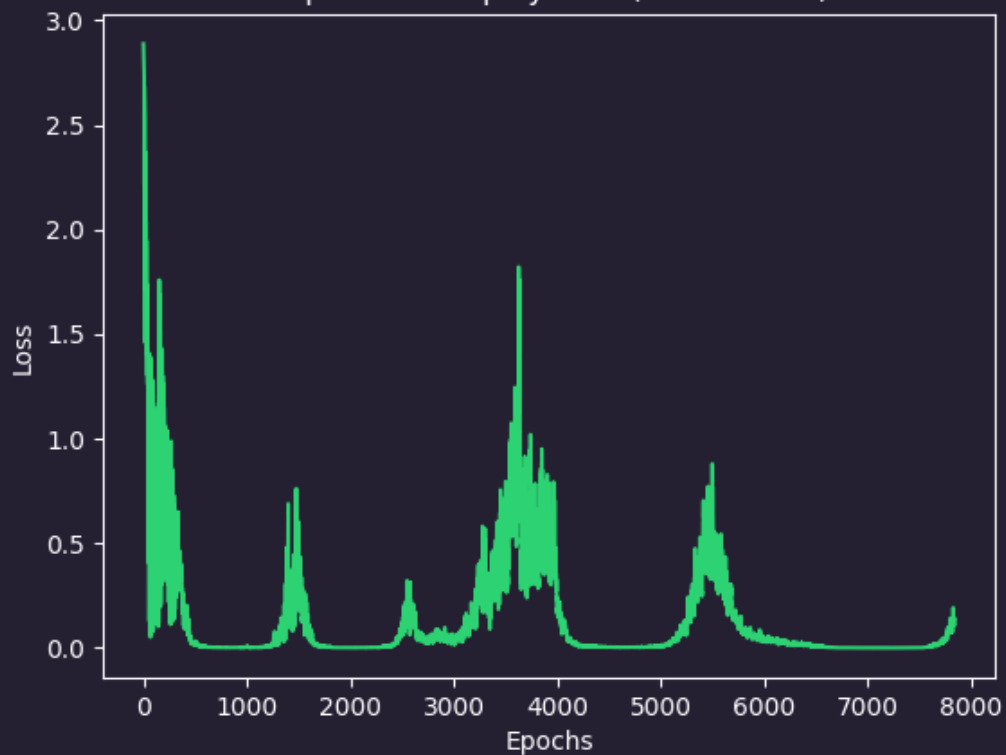
Result 1:

Naive DQN Loss (Static Mode)



Naive DQN Loss

Experience Replay Loss (Static Mode)



Experience Replay DQN Loss

? Q2: HW3-2 Enhanced DQN Variants for player mode

Question / Task:

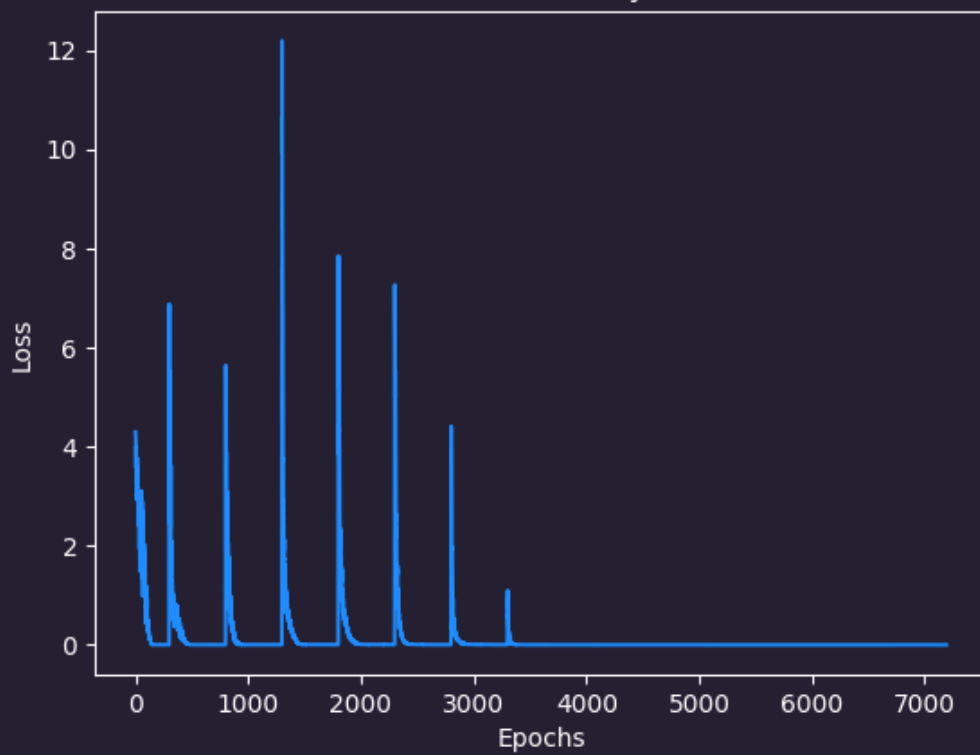
Implement and compare Double DQN and Dueling DQN. Focus on how they improve upon the basic DQN approach.

A2 (Improvements Explanation):

- **Double DQN:** Basic DQN suffers from overestimation bias because it uses the same network to select and evaluate actions. Double DQN decouples this: it uses the Main Network to select the action and the Target Network to evaluate it, yielding more accurate Q-values.
- **Dueling DQN:** Splits the network into a Value stream (how good the state is generally) and an Advantage stream (how much better an action is compared to others). This allows the network to learn which states are valuable without having to learn the effect of each action for every single state.

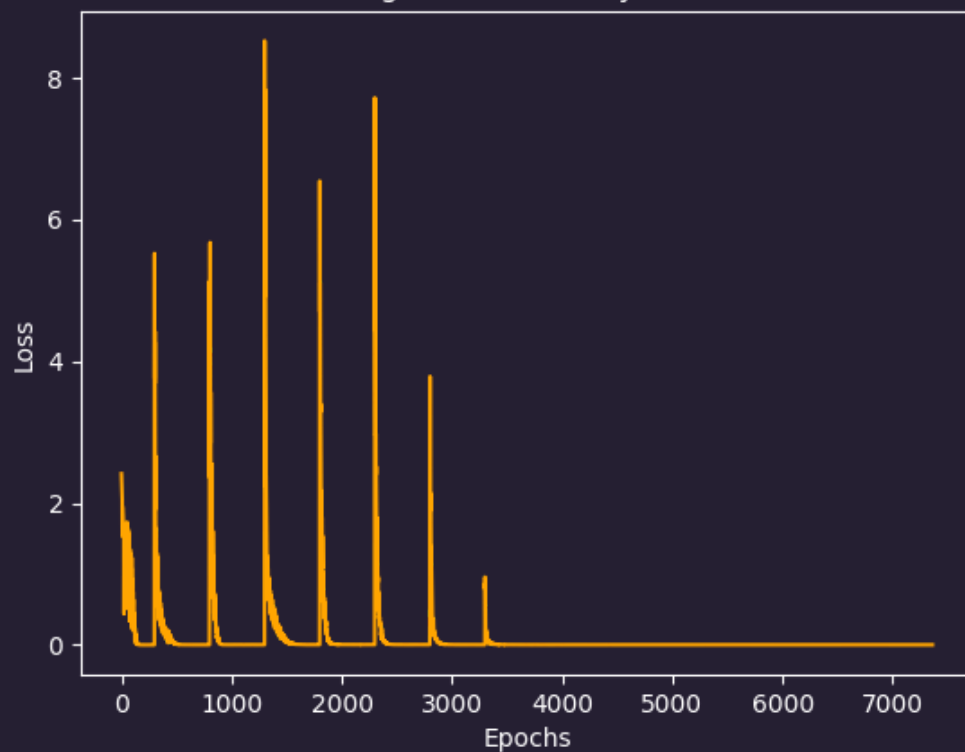
Result 2:

Double DQN Loss (Player Mode)



Double DQN Loss

Dueling DQN Loss (Player Mode)



Dueling DQN Loss

? Q3: HW3-3 Enhance DQN for random mode WITH Training Tips

Question / Task:

Convert the DQN model from PyTorch to either Keras or PyTorch Lightning. Bonus points for integrating training techniques to stabilize/improve learning.

A3 (Implementation & Tips):

We converted the model to **PyTorch Lightning** (`LitDQN` class) for the hardest `random` environment. To stabilize training in this chaotic mode, we integrated:



Huber Loss:

Replaced MSE with

```
nn.SmoothL1Loss()
```

to prevent exploding gradients when rewards fluctuate wildly.



Learning Rate Scheduling:

Added

```
StepLR
```

to gradually decay the learning rate, helping the model fine-tune its policy as it converges.



Gradient Clipping:

Set

```
gradient_clip_val=1.0
```

in the Trainer to ensure updates remain within a stable bound.

Result 3:

The PyTorch Lightning model successfully encapsulated the training loop. By applying these tips, the model avoided catastrophic forgetting in the `random` environment where

all objects (Player, Goal, Pit, Wall) spawn arbitrarily, successfully converging across epochs.